

Algoritma Analizi ve Temeller

Hafta 1



<https://gemini.google.com/share/52633b92ef65>

Algoritma Nedir?

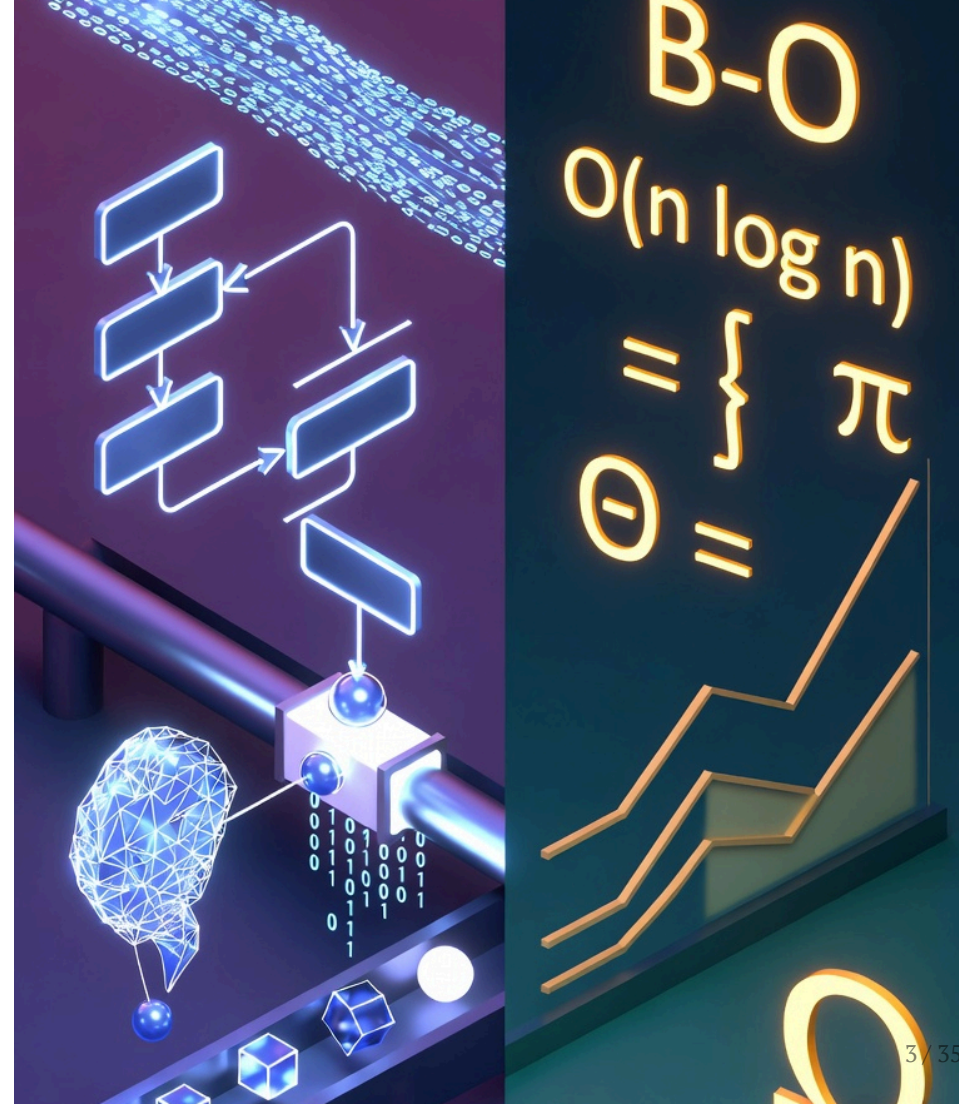
- Algoritma: Problemleri çözmek için tasarlanan kesin ve sonlu adımlar bütünüdür.
- Girdi verilerini işleyerek mantıksal adımlarla anlamlı bir çıktıya dönüştürür.
- Temel kriterler: Kesinlik, sonluluk, etkinlik ve tanımlı bir girdi-çıkı kümesi.
- Hesaplamalı düşünme sürecinin mantıksal tasarımı ve uygulama yol haritasıdır.

<https://gemini.google.com/share/52633b92ef65>

Algoritma Analizi Motivasyonu

- Sınırlı kaynakların (zaman ve bellek) en verimli şekilde yönetilmesi gerekliliği.
- Büyük veri setlerinde küçük verimlilik farklarının devasa zaman kayıplarını önlemesi.
- Donanım ve programlama dilinden bağımsız, evrensel performans ölçütleri geliştirme.
- Farklı çözüm yollarını matematiksel olarak kıyaslayıp en iyisini seçme becerisi.

<https://gemini.google.com/share/52633b92ef65>



RAM (Random Access Machine) Modeli

- Analiz için kullanılan, gerçek donanımdan soyutlanmış ideal bir bilgisayar modeli.
- Temel operasyonların (aritmetik, atama) sabit bir birim zaman aldığı varsayılır.
- Hafızadaki her hücreye erişim maliyeti eşit ve sabit ($O(1)$) kabul edilir.
- Karmaşıklığı sadece algoritmik adımlara indirgeyen standart bir laboratuvar ortamıdır.

<https://gemini.google.com/share/52633b92ef65>

Zaman Karmaşıklığı vs Bellek Karmaşıklığı

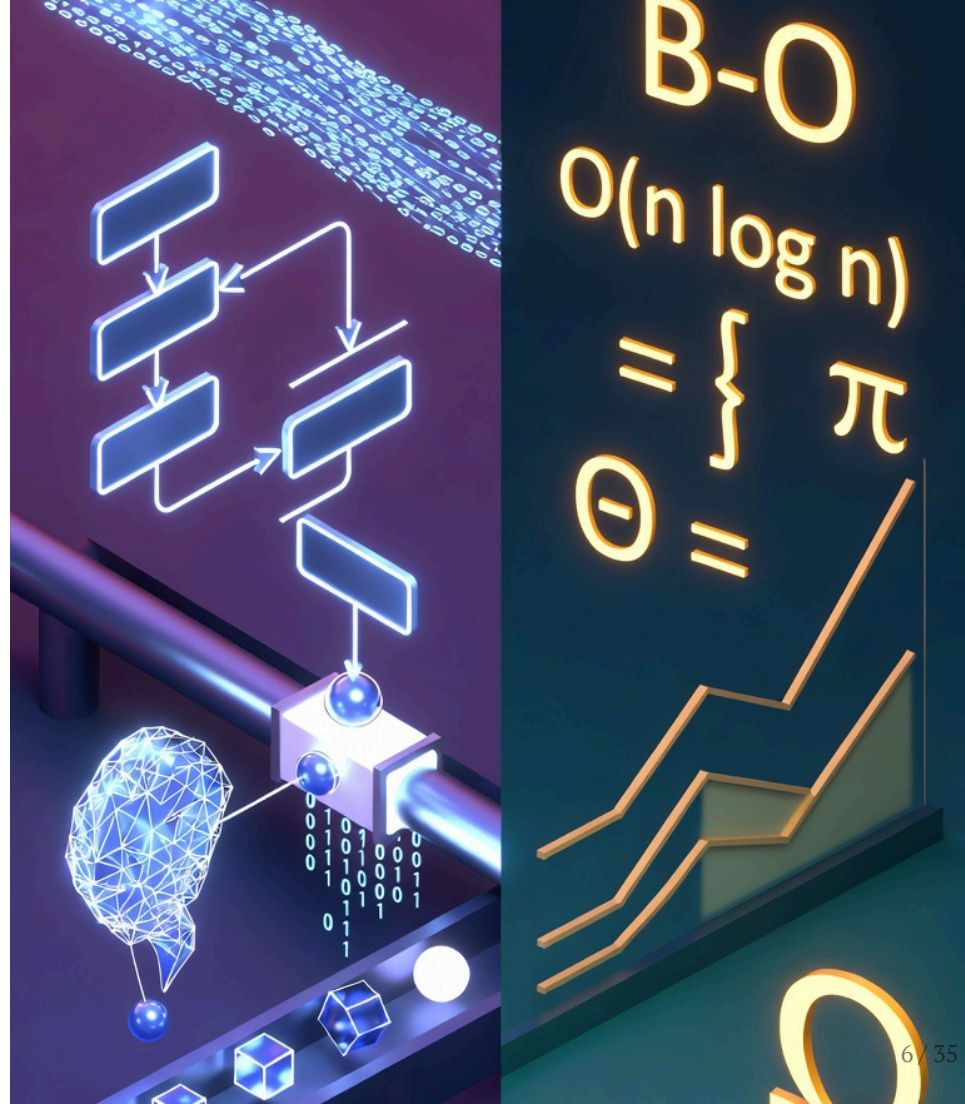
- Zaman Karmaşıklığı: Giriş boyutu arttıkça temel operasyon sayısının artış hızı.
- Bellek Karmaşıklığı: Algoritmanın çalışması için gereken ekstra depolama alanı miktarı.
- Zaman-Bellek Takası: Hız kazanmak için daha fazla bellek kullanma stratejisi.
- Modern sistemlerde genellikle zaman tasarrufu, bellek tasarrufundan daha önceliklidir.

<https://gemini.google.com/share/52633b92ef65>

Giriş Boyutu (Input Size) Kavramı

- Analizin temel değişkeni olan 'n' değeri, verinin büyüklüğünü temsil eder.
- n'in anlamı probleme göre değişir: dizi uzunluğu, düğüm sayısı, bitler.
- Küçük n değerlerinde her algoritma hızlıdır; asıl fark n büyüdüğünde çıkar.
- Algoritmanın ölçeklenebilirliğini ölçmek için n değerinin sonsuza gidişi baz alınır.

<https://gemini.google.com/share/52633b92ef65>



Büyüme Oranı (Rate of Growth)

- Fonksiyonun giriş boyutu (n) arttıkça sergilediği baskın davranış biçimidir.
- Sabit katsayılar ve düşük dereceli terimler, büyük n değerlerinde önemsizleşir.
- Hiyerarşi: Sabit < Logaritmik < Lineer < Karesel < Üstel büyüme modelleri.
- Darboğazların tespiti için en yüksek dereceli terimin etkisine odaklanılır.

<https://gemini.google.com/share/52633b92ef65>

Complexity Growth Visualizer

Input Size (n): 10

☒ $O(1)$ ☒ $O(\log n)$ ☒ $O(n)$ ☒ $O(n \log n)$ ☒ $O(n^2)$

Reset

Play

<https://gemini.google.com/share/52633b92ef65>

En Kötü, En İyi ve Ortalama Durum

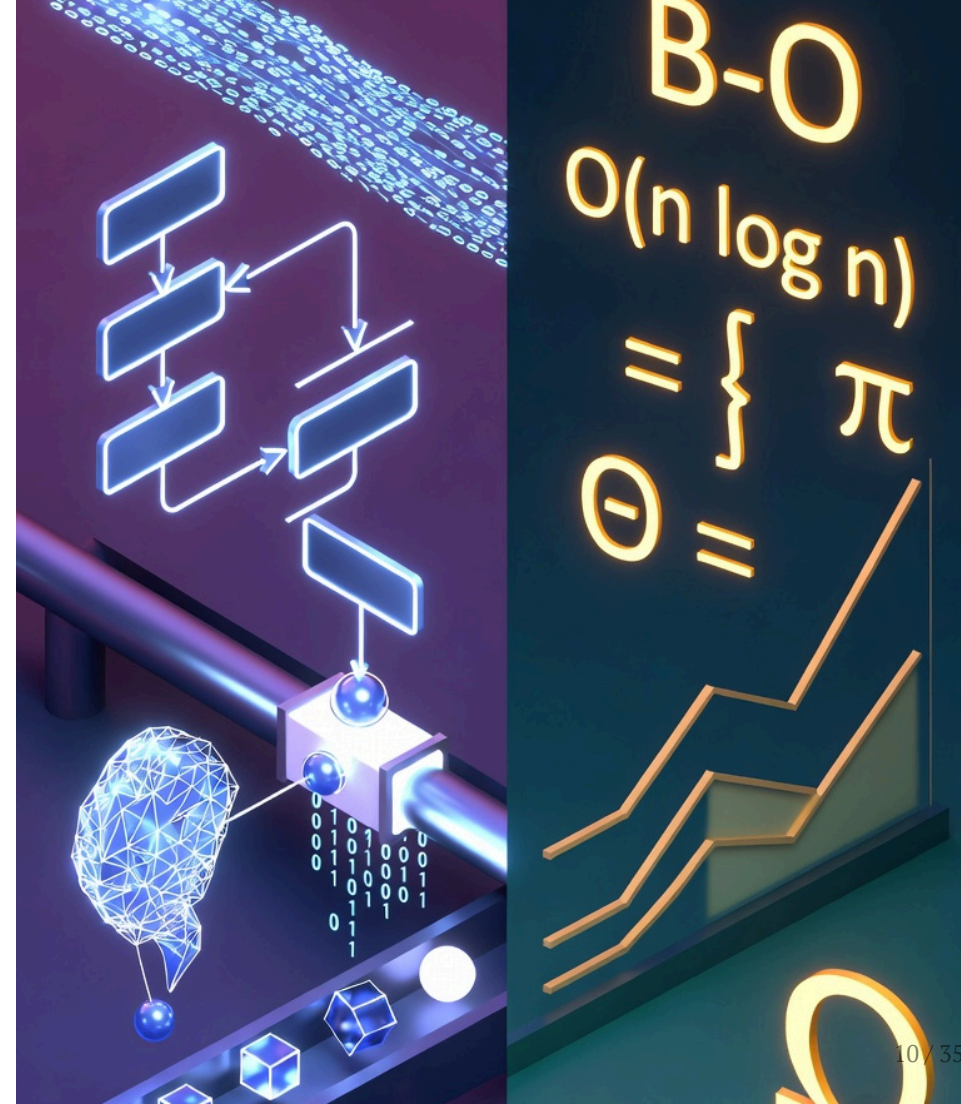
- Worst-case (En Kötü): Algoritmanın verebileceği maksimum garanti ve üst sınır.
- Best-case (En İyi): Verinin en ideal dizilimle gelmesi durumu.
- Average-case (Ortalama): Olasılık dağılımına göre beklenen genel performans değeri.
- Mühendislikte her zaman en kötü durum analizi ile sistem güvenliği sağlanır.

<https://gemini.google.com/share/52633b92ef65>

Asimptotik Notasyonlara Giriş

- Karmaşıklığı sınıflandırmak için kullanılan evrensel ve standart matematiksel dildir.
- Fonksiyonun tam değerinden ziyade büyüme karakterine ve 'şekline' odaklanır.
- Ayrıntıları temizleyerek algoritmalar arası adil bir kıyaslama imkanı sunar.
- Big-O, Big-Omega ve Big-Theta notasyonları bu dilin temel taşlarıdır.

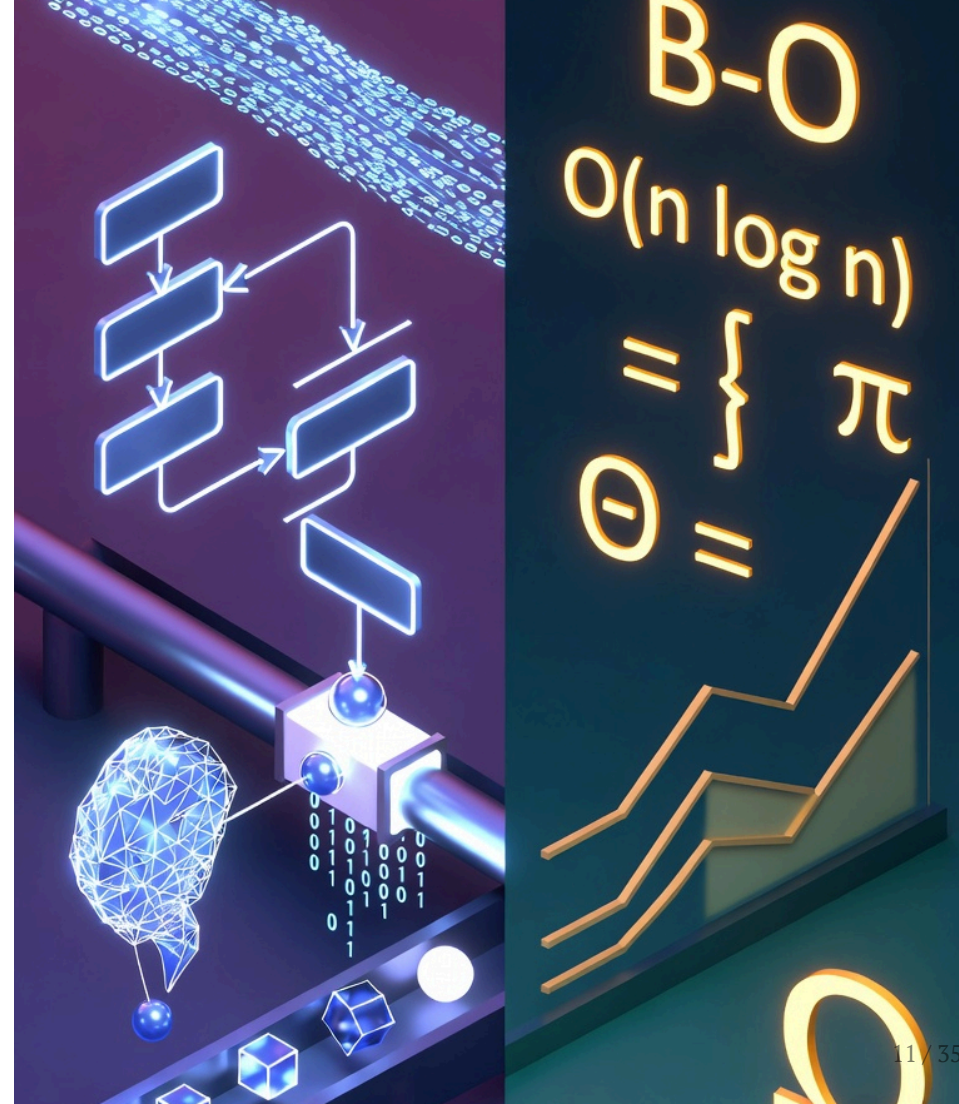
<https://gemini.google.com/share/52633b92ef65>



Big-O (O) Notasyonu Matematiksel Tanımı

- Asimptotik Üst Sınır: Algoritmanın çalışma süresi bu sınırı asla geçemez.
- Matematiksel tanım: $f(n) \leq c \cdot g(n)$ koşulunun tüm $n > n_0$ için sağlanması.
- Performans için bir tavan fiyat belirleyerek maksimum maliyeti garanti eder.
- Algoritmanın 'bundan daha yavaş çalışmayacağı' güvencesini veren mühendislik ölçütüdür.

<https://gemini.google.com/share/52633b92ef65>



Big-O Notasyonu Örnekleri

- $O(1)$ Sabit Zaman: Girdi boyutundan bağımsız, anlık erişim (indislerle eleman okuma).
- $O(\log n)$ Logaritmik Zaman: Her adımda veri setini yarıya indirme (Binary Search).
- $O(n)$ Lineer Zaman: Her elemanı bir kez kontrol etme (Basit arama).
- $O(n^2)$ Karesel Zaman: Her elemanı diğer her elemanla kıyaslama (İç içe döngüler).

<https://gemini.google.com/share/52633b92ef65>

Big-Omega (Ω) Notasyonu Matematiksel Tanımı

- Asimptotik Alt Sınır: Algoritmanın gerektirdiği minimum iş yükünü temsil eder.
- Matematiksel tanım: $f(n) \geq c \cdot g(n)$ koşulunun tüm $n > n_0$ için sağlanması.
- Problemin doğasından gelen kaçınılmaz maliyetlerin ve teorik limitlerin belirlenmesi.
- Algoritmanın 'en az şu kadar iş yapacağı' garantisini sunar.

Big-Omega Notasyonu Örnekleri

- $\Omega(n)$: Bir dizideki elemanların toplamını bulmak için tüm verilere bakılmalıdır.
- $\Omega(n \log n)$: Karşılaştırma tabanlı sıralama algoritmalarının ulaşabileceği teorik alt sınır.
- Alt sınırın bilinmesi, algoritma optimizasyon çalışmalarında duracağımız noktayı belirler.
- Problemlerin zorluk derecesini ölçmek için kullanılan temel matematiksel araçtır.

<https://gemini.google.com/share/52633b92ef65>

Big-Theta (Θ) Notasyonu Matematiksel Tanımı

- Asimptotik Kesin Sınır: Hem üst hem alt sınırın çakıştığı durumdur.
- Algoritmanın karakteristiğini bir bant aralığında tam olarak hapseder.
- Matematiksel olarak: $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ şartıdır.
- Analizdeki en değerli, en kesin ve en bilgilendirici notasyon türüdür.

<https://gemini.google.com/share/52633b92ef65>

Big-Theta Notasyonu Örnekleri

- $\Theta(n)$: Bir diziye baştan sona tam bir kez tarayan standart döngüler.
- $\Theta(n^2)$: Her koşulda iç içe n defa dönen matris işlemleri.
- Düşük ve yüksek sınırların birbirine çok yakın olduğu kararlı durumlar.
- Deterministik algoritmaların performansını ifade etmede en sağlıklı yöntemdir.

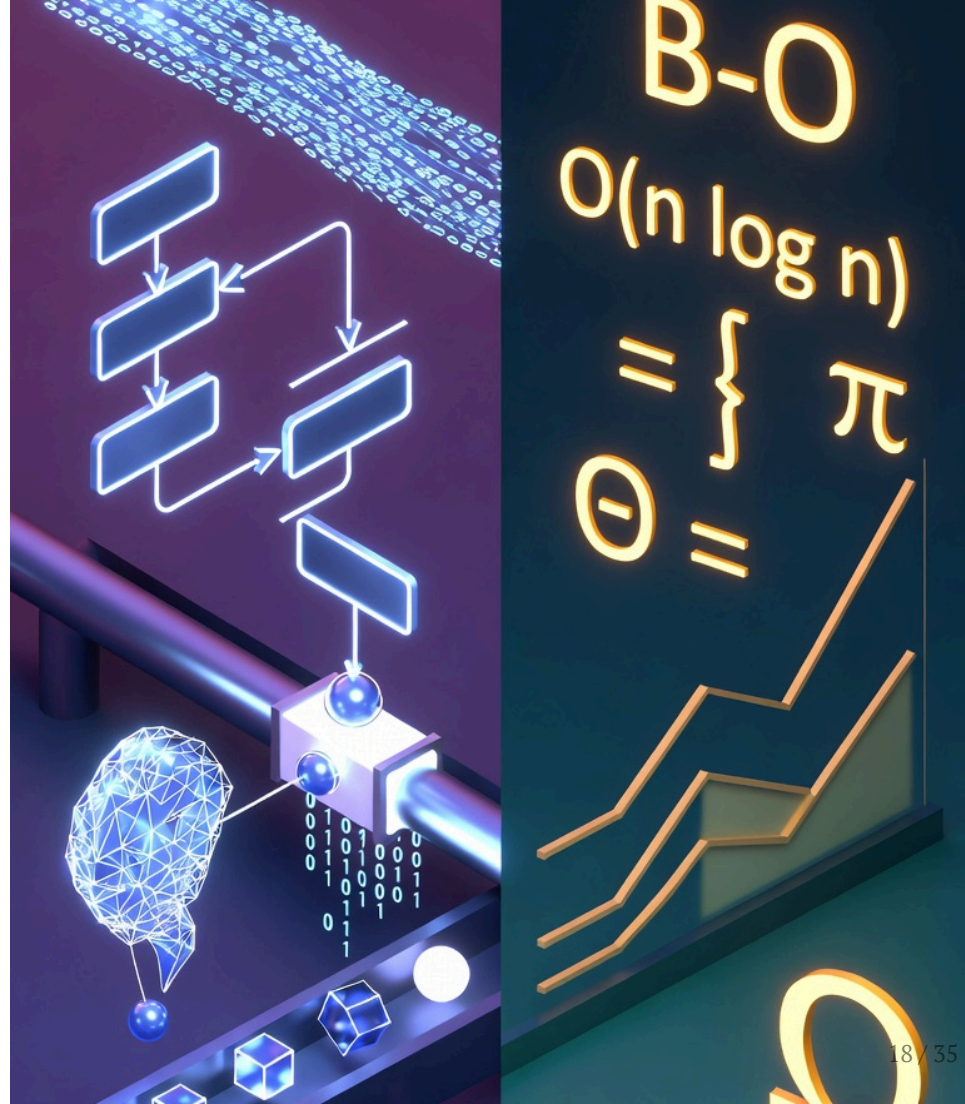
Little-o ve Little-omega Notasyonları

- Strictly (Kesin) Sınırlar: Eşitlik durumunun asla sağlanmadığı asimptotik farklar.
- Little-o: $f(n)$ fonksiyonunun $g(n)$ yanında önemsiz kalması (limit = 0).
- Little-omega: $f(n)$ fonksiyonunun $g(n)$ karşısında sonsuzlaşması (limit = sonsuz).
- Fonksiyonlar arasındaki hiyerarşiyi ve baskınlık derecesini matematiksel ispatlarla gösterir.

Döngülerin Analizi (Tekli Döngüler)

- Basit döngülerde karmaşıklık, toplam iterasyon sayısına doğrudan bağlıdır.
- Döngü içindeki sabit zamanlı işlemler, toplam süreyi etkilemez ($O(1)$).
- Artış miktarı lineerse $O(n)$, geometrikse ($i = i*2$) $O(\log n)$ olur.
- Kodun kalbini oluşturan bu yapıların analizi, performans optimizasyonunun ilk adımıdır.

<https://gemini.google.com/share/52633b92ef65>

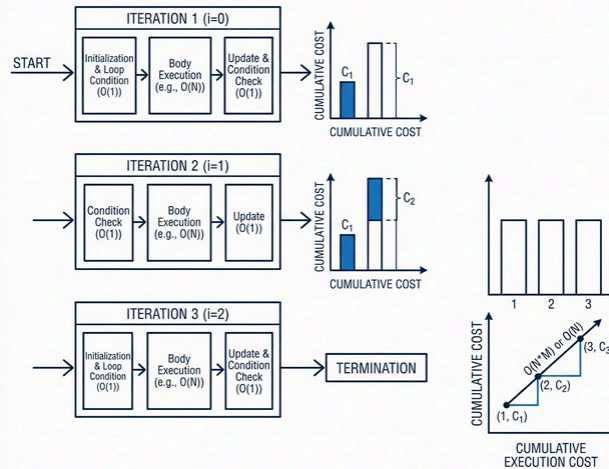


Algorithm

```
1  procedure sum_array(A):  
2      sum = 0  
3      for i = 1 to length(A):  
4          sum = sum + A[i]  
5      return sum
```

<https://gemini.google.com/share/52633b92ef65>

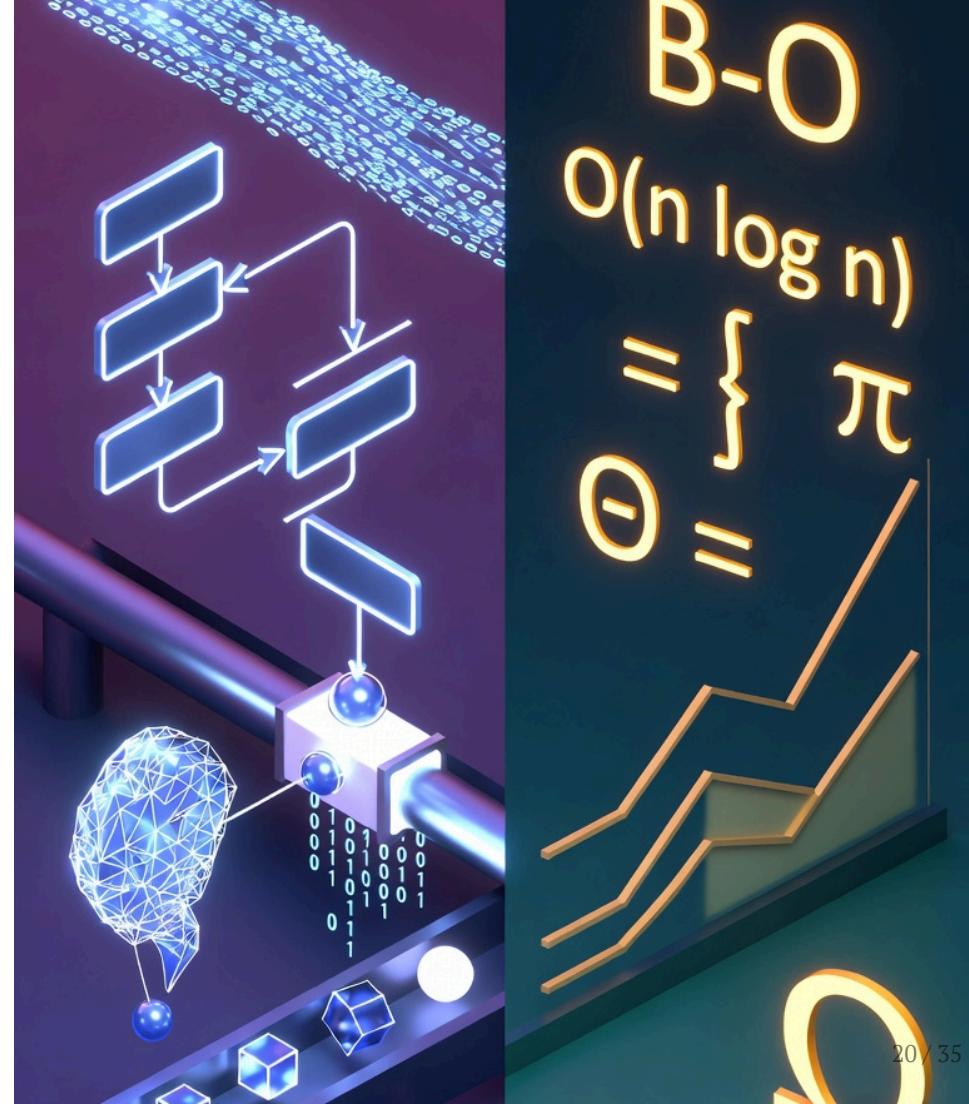
ITERATIVE LOOP EXECUTION COST VISUALIZATION



İç İçe Döngülerin Analizi

- Nested Loops: Dış döngünün her adımı için iç döngünün tetiklenmesi.
- Bağımsız döngülerde karmaşıklık, döngü boyutlarının çarpımıdır ($n * m$).
- Bağımlı döngülerde ($j < i$) aritmetik dizi toplamı formülleri devreye girer.
- Üçüncü dereceden (n^3) ve daha yüksek karmaşıklıklar genellikle verimlilik kaybıdır.

<https://gemini.google.com/share/52633b92ef65>



Algorithm

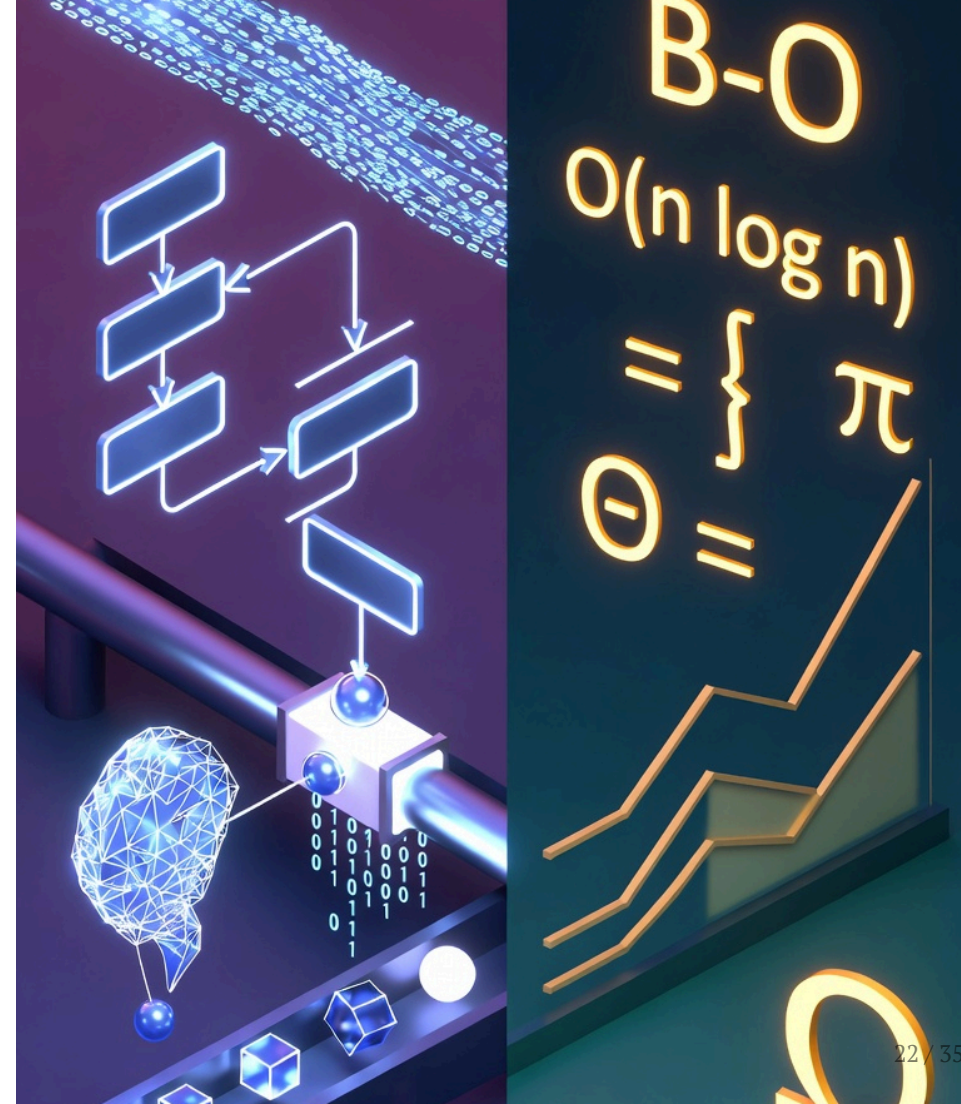
```
1  procedure nested_example(n):  
2      count = 0  
3      for i = 1 to n:  
4          for j = 1 to n:  
5              count = count + 1
```

<https://gemini.google.com/share/52633b92ef65>

Ardışık İfadelerin Analizi

- Kod içinde peş peşe gelen blokların karmaşıklığı toplanarak hesaplanır.
- Toplam Karmaşıklık: Asimptotik olarak en yüksek dereceli bloğun baskınlığıdır.
- Kural: $T(n) = O(f(n) + g(n)) = O(\max(f(n), g(n)))$.
- Düşük maliyetli bloklar, büyük veri setlerinde asimptotik olarak ihmal edilir.

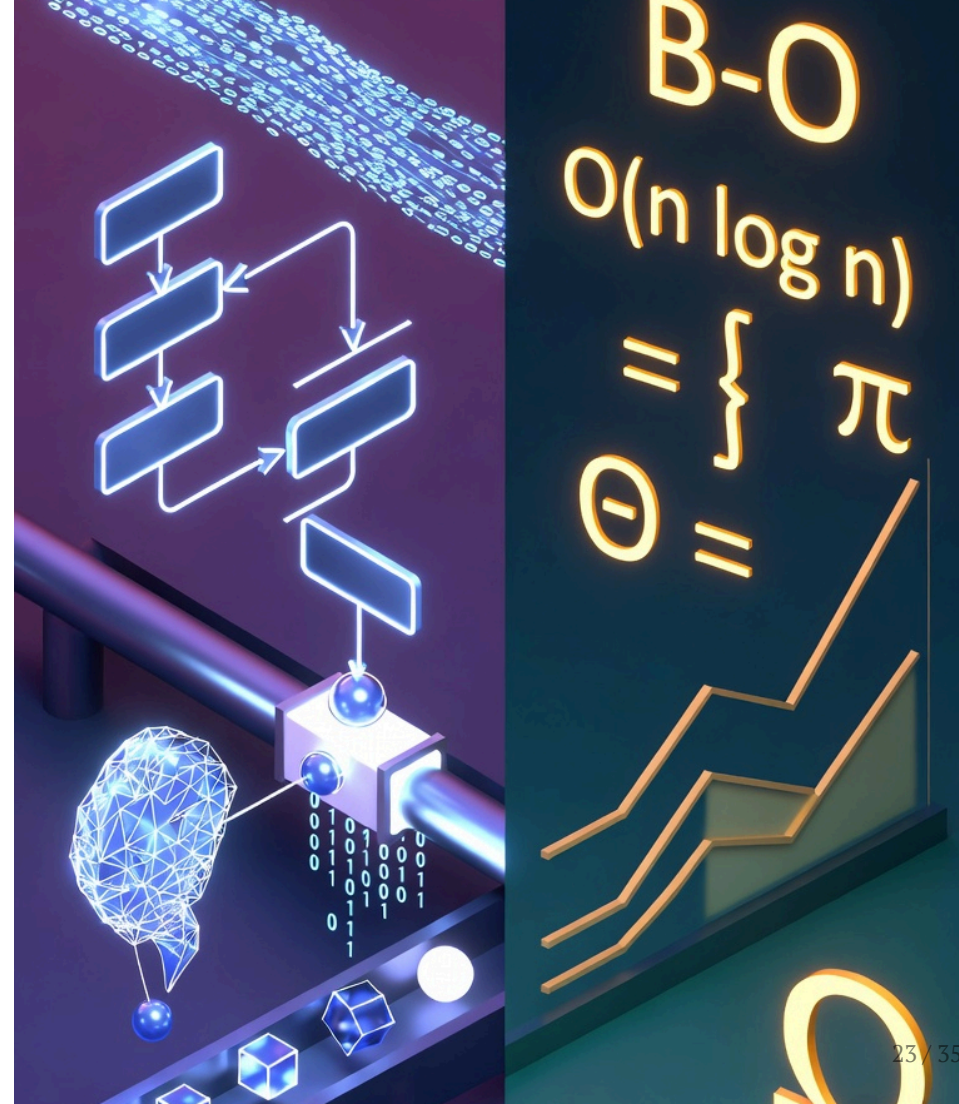
<https://gemini.google.com/share/52633b92ef65>



Koşullu İfadelerin Analizi (If-Else)

- Branching: Hangi dalın seçileceği çalışma anında veriye göre belirlenir.
- Worst-case analizi için en maliyetli (en yavaş) dal baz alınır.
- Koşul kontrolünün kendi maliyeti ($O(1)$) her zaman toplama eklenir.
- Dallanma tahmini (Branch Prediction) gibi donanım etkileri teorik analizde ihmal edilir.

<https://gemini.google.com/share/52633b92ef65>



Algorithm

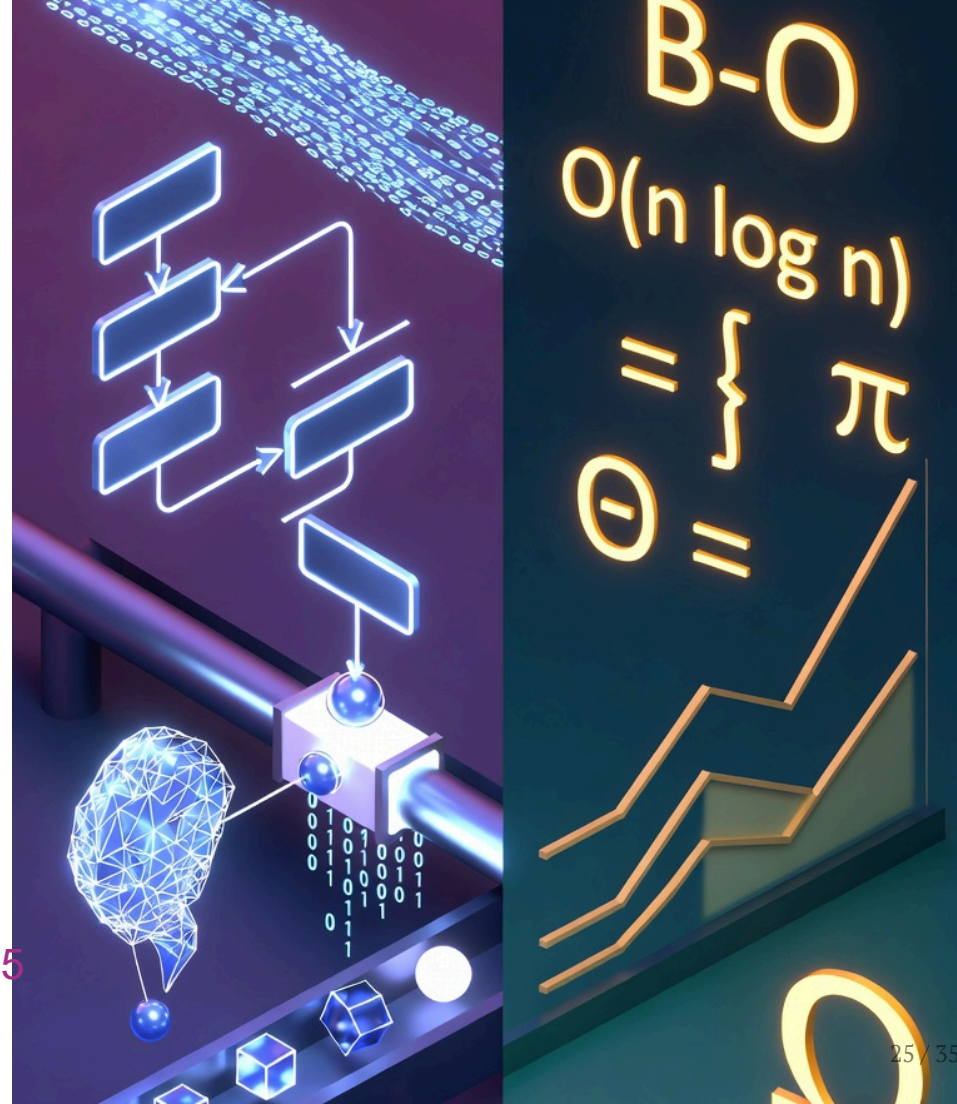
```
1  if condition:
2      O(n^2) block
3  else:
4      O(n) block
```

<https://gemini.google.com/share/52633b92ef65>

Özyineleme (Recursion) Kavramı ve Analizi

- Problemi kendi alt parçalarına bölerek çözen fonksiyonel derinlik yapısıdır.
- Base Case (Sonlanma) adımı, sonsuz döngüleri engelleyen hayati emniyet kilididir.
- Call Stack: Her çağrı bellekte yer kaplar (bellek karmaşıklığı riski).
- Karmaşıklık analizi, 'Rekürans Bağlılıkları' adı verilen özel denklemlerle yapılır.

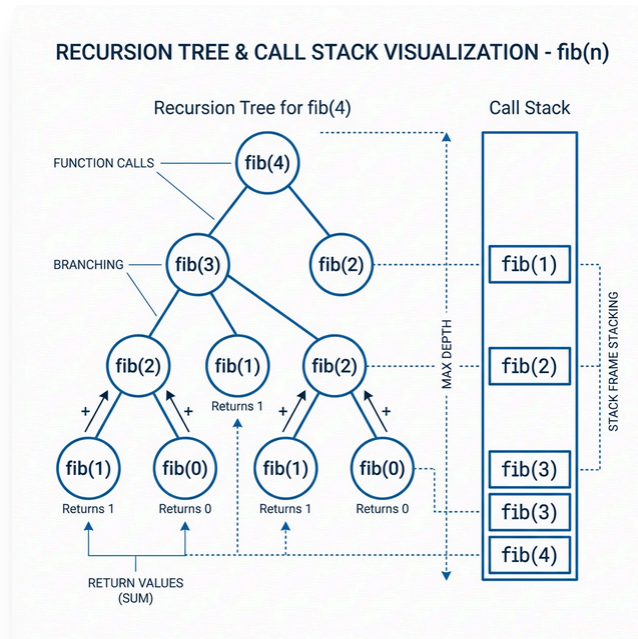
<https://gemini.google.com/share/52633b92ef65>



Algorithm

<https://gemini.google.com/share/52633b92ef65>

```
1  procedure factorial(n):  
2      if n == 0 or n == 1:  
3          return 1  
4      return n * factorial(n - 1)
```



Recursion Tree: Factorial(6)

Step Backward

Step Forward

Reset

Step: 1 / 12

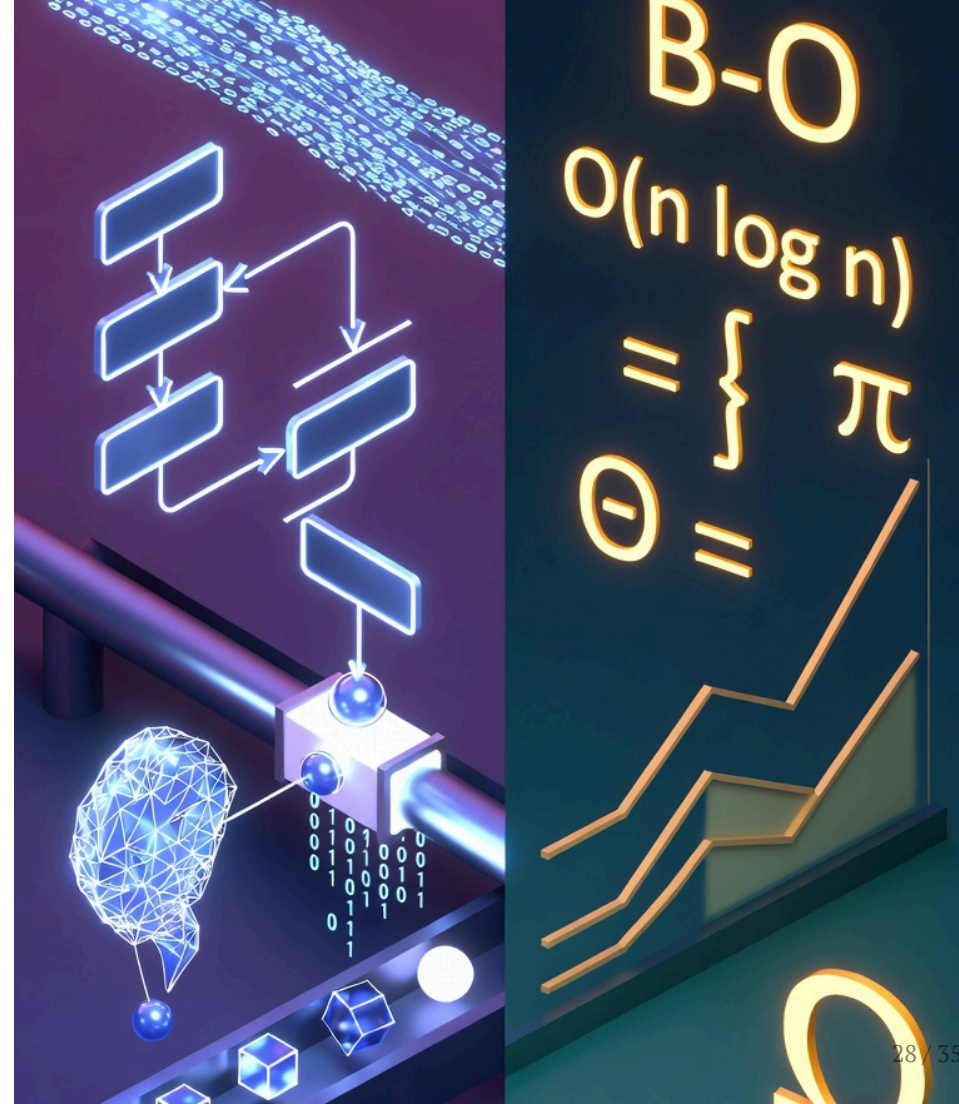
Current Action: Calling Factorial(6)...

<https://gemini.google.com/share/52633b92ef65>

Rekürans Bağıntıları (Recurrence Relations)

- Özyinelemeli algoritmaların çalışma süresini ifade eden matematiksel denklemlerdir.
- Genel format: $T(n) = aT(n/b) + f(n)$ (bölme, özyineleme ve ek iş).
- Çözüm yöntemleri: Yerine koyma, Ağaç yöntemi ve Master teoremi.
- Algoritmanın ne kadar hızlı 'küçüldüğünü' ve toplam maliyetini belirler.

<https://gemini.google.com/share/52633b92ef65>



Master Teoremi (Master Theorem) Tanımı

- Özyinelemeli denklemleri hızlıca çözmek için kullanılan genel bir şablon formülüdür.
- $T(n) = aT(n/b) + n^d$ yapısındaki denklemlere doğrudan uygulanabilir.
- a: alt problem sayısı, b: bölünme oranı, d: birleştirme maliyeti derecesi.
- İş yükünün ağacın neresinde (kök mü, yaprak mı) yoğunlaştığını tespit eder.

<https://gemini.google.com/share/52633b92ef65>

Master Teoremi Durum 1 ve Durum 2

- Durum 1: İş yükünün yapraklarda (en alt seviyede) yoğunlaştığı senaryolardır.
- Durum 2: İş yükünün her seviyede dengeli dağıldığı (Merge Sort gibi) durumlardır.
- Dengeli dağılımda genellikle ortaya bir logaritmik çarpan ($n \log n$) eklenir.
- Algoritmanın bölünme hızı ile iş yapma hızı arasındaki kritik dengeyi ölçer.

<https://gemini.google.com/share/52633b92ef65>

Master Teoremi Durum 3 ve Örnek Çözümler

- Durum 3: Ana işin bölme/birleştirme (kök) seviyesinde baskın olduğu durumlardır.
- Bu durumda algoritmanın toplam karmaşıklığı doğrudan ek işe (n^d) eşittir.
- Regülerite şartı: Alt problemlerin her adımda gerçekten küçüldüğünün matematiksel ispatı.
- Teoremin uygulanamadığı limit durumlar (polinomial olmayan farklar) analiz dışıdır.

<https://gemini.google.com/share/52633b92ef65>

Hafta 1 Genel Özeti ve Kapanış

- Algoritma analizinin matematiksel ve mantıksal temellerini inşa ettik.
- Asimptotik notasyonlar (O , Ω , Θ) ile performans ölçüm standartlarını kavradık.
- Döngüsel ve özyinelemeli yapıların maliyet hesaplama tekniklerini öğrendik.
- Gelecek Hafta: Bu teorik temelleri 'Sıralama Algoritmaları' üzerinde uygulayacağız.

<https://gemini.google.com/share/52633b92ef65>

Soru 1

$T(n) = 3T(n/2) + O(n)$ şeklinde verilen bir özyinelemeli fonksiyonun zaman karmaşıklığını Master Teoremi kullanarak bulunuz.

Cevap Adımları

- $a = 3, b = 2, d = 1$ olarak belirlenir
- $\log_b(a) = \log_2(3)$ değeri hesaplanır (yaklaşık 1.58)
- $\log_2(3) > 1$ (d) olduğu için Durum 1 uygulanır
- Sonuç: $O(n^{\log_2(3)})$ olarak bulunur

Learning Outcome: Master Teoremi Durum 1 analizi

Soru 2 (Orta Seviye)

İki iç içe `for` döngüsü düşünün: dıştaki döngü 1'den n 'e kadar, içteki döngü ise 1'den i 'ye kadar çalışmaktadır. Bu kod bloğunun Big-O zaman karmaşıklığı nedir?

Çözüm Adımları

- Dış döngü n defa çalışır.
- İç döngü her i değeri için i defa çalışır (Dışa bağımlı döngü).
- Toplam adım sayısı: $\sum_{i=1}^n i = 1 + 2 + 3 + \dots + n$
- Aritmetik dizi toplamı: $\frac{n(n+1)}{2} = \frac{n^2+n}{2}$
- **Sonuç:** Asimptotik olarak en büyük terim n^2 olduğundan karmaşıklık $O(n^2)$ 'dir.

Learning Outcome: Bağımlı iç içe döngülerin asimptotik analizi

Soru 3 (Zor)

Bir algoritmanın zaman maliyeti $T(n) = T(n/2) + T(n/4) + O(n)$ olarak modellenmiştir. Bu algoritmanın üst sınırı (Big-O) nedir ve Master Teoremi doğrudan uygulanabilir mi?

Çözüm Adımları

- **Master Teoremi** bu formata doğrudan uygulanamaz; çünkü alt problemlerin bölünme oranları (b değerleri: 2 ve 4) birbirinden farklıdır.
- Çözüm için **Özyineleme Ağacı (Recursion Tree)** yöntemi kullanılır.
- Her seviyedeki maliyet toplamı: Kökte cn , bir alt seviyede $c(n/2 + n/4) = c(3n/4)$.
- Seviye maliyetleri azalan bir geometrik seri oluşturur: $\sum_{i=0}^{\infty} cn \cdot \left(\frac{3}{4}\right)^i$
- Toplam maliyet: $cn \cdot \frac{1}{1-3/4} = 4cn$
- **Sonuç:** Sabit çarpanlar ihmal edildiğinde karmaşıklık $O(n)$ olarak bulunur.



Learning Outcome: Asimetrik özyineleme ağaçlarının analizi